

FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of Mon 18 JUNE 2018 - Part Theory

LAST + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

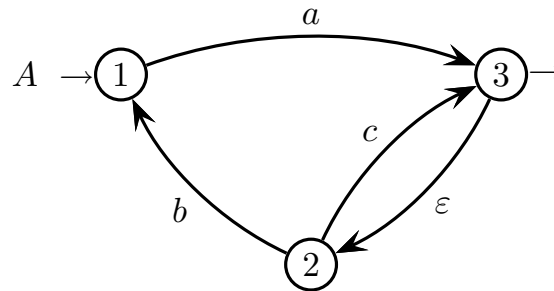
SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): Syntax and Semantics of Languages
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass part theory, the candidate must answer the mandatory (not optional) questions; notice that the full grade is achieved by answering the optional questions.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.15m

1 Regular Expressions and Finite Automata 20%

1. Consider the nondeterministic automaton A over the three-letter alphabet $\{a, b, c\}$:



Answer the following questions:

- (a) List all the strings of length from 0 to 4 (0 and 4 included) that are accepted by automaton A . Say if automaton A is ambiguous or not and explain why.
 - (b) Cut the spontaneous transition of automaton A , so that after cutting the result is deterministic. Say if the result is minimal and if necessary minimize it (call it A'). Verify the correctness of A' with the short strings of point (a).
 - (c) Through node elimination (Brzozowsky), write a regular expression R equivalent to automaton A . Verify the correctness of R with the short strings of point (a).
 - (d) Through the Berry-Sethi method, determinize automaton A (nondeterministic), and obtain a (deterministic) automaton A'' . If necessary minimize automaton A'' and verify that it is identical to automaton A' .
 - (e) (optional) Say if the regular language $L(A)$ is local or not and explain why.
-

2 Free Grammars and Pushdown Automata 20%

1. Consider the context-free language L defined below, over the three-letter alphabet $\Sigma = \{ a, b, c \}$:

$$L = \{ y = x c x^R \mid x \in \{ a, b \}^+ \text{ and } |y| = 3k \text{ for some } k \geq 1 \}$$

Examples:

$a c a$ $a b a b c b a b a$

Counterexamples:

$a b b a$ $a b c b a$ $a b a b c a b a b$

Answer the following questions:

- (a) Write the four smallest possible values for the length $|y|$ of the phrases of language L , and the corresponding values of $|x|$.
- (b) Write a non-ambiguous *BNF* grammar G that generates language L . Solutions with a small number of nonterminal symbols and rules are preferred.
- (c) Draw the syntax trees of the following two phrases of language L :

$a c a$ $a b a b c b a b a$

2. Consider a language of a special kind of arithmetic expression, specified as follows:

- there are two *binary* operators, addition `+` and multiplication `*`
- addition and multiplication are *prefix* operators, i.e., `+` `a a` and `*` `a a`
- there is also a *n*-ary operator `max (a1, a2, ..., an)`, with $n \geq 2$, which returns the greatest of its arguments `ai` (for $1 \leq i \leq n$)
- the maximum operator is idempotent, for instance it holds:

$$\text{max (max (a, b), max (c, d))} = \text{max (a, b, c, d)}$$

thus for optimization a (sub)expression like the one below:

$$\text{max (max (...), max (...), \dots, max (...))}$$

is considered *invalid* and *is not* in the language

- an expression operand is a signed integer decimal number without leading zeroes, or an alphanumerical identifier possibly with dash (dashes may be consecutive), starting with a letter and not ending with a dash (letters are only lowercase)

For any unspecified minor detail, see the samples below (all of them have a schematic operand `a`). You may also follow the usual conventions, e.g., those of the C language.

A few valid expressions (think of how they are computed before rushing to answer):

`+ a a`

`* a a`

`+ * a a a`

`+ a * a a`

`+ + a max (a, a) a`

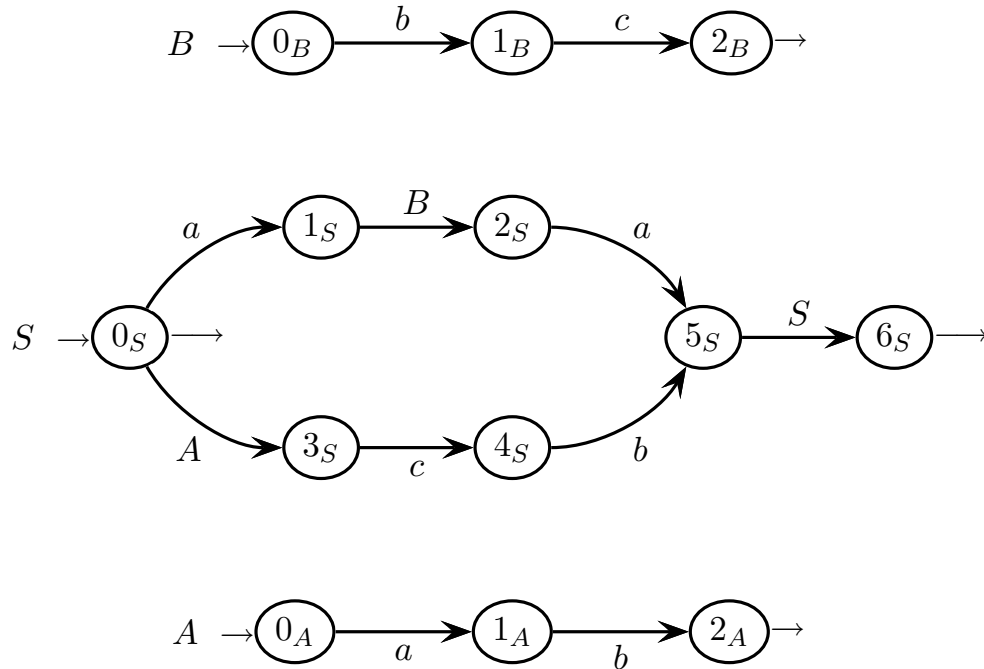
`max (+ * a a a, max (+ a a, a))` DRAW THE SYNTAX TREE OF THIS EXPR

This is a counterexample: `max (max (a, + a a), max (* a a, a))`.

Write a non-ambiguous grammar, in general of type *EBNF*, that models the language of expressions described above. Verify the correctness of your grammar by drawing the syntax tree of the last valid sample expression.

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the following grammar G , represented as a machine net over the three-letter terminal alphabet $\Sigma = \{ a, b, c \}$ and the three-letter nonterminal alphabet $V = \{ S, A, B \}$ (axiom S):



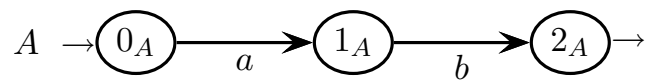
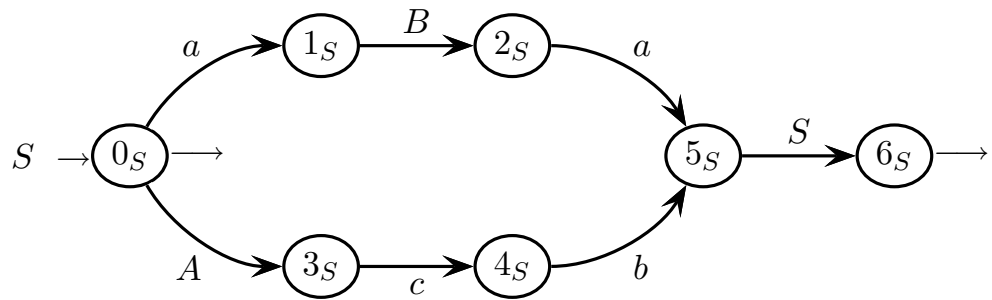
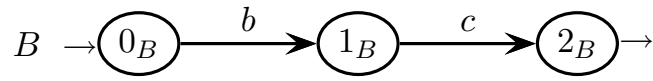
Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) Draw the complete pilot of grammar G and prove that grammar G is not of type $ELR(1)$. Explain your answer.
- (b) Modify grammar G , as little as possible (keep the same nonterminal alphabet), so as to obtain an equivalent grammar (generating the same language) that is of type $ELR(1)$. Motivate your choice.
- (c) Write all the guide sets on the shift arcs, call arcs and exit arrows of the machine net of grammar G , and show that grammar G is not of type $ELL(1)$. Explain your answer.
- (d) Determine whether grammar G is of type $ELL(k)$, for some $k > 1$. Explain your answer.
- (e) (optional) Which is the smallest language family, among those studied in the present course, that includes language $L(G)$? Motivate your answer.

please here draw the pilot – question (a)

please here draw the net of the modified grammar – question (b)

please here draw the call arcs and annotate the guide sets – question (c)



please here answer questions (d) and (e)

free space left for the continuation of any question of any exercise

4 Language Translation and Semantic Analysis 20%

1. Consider a source language L of non-empty nested lists of arbitrary depth. A list element may be an atom or a sublist. An atom is schematized by a terminal “e”. A sublist is enclosed between round brackets “()”, but the main list is not. Each element (atom or sublist), at any depth, has a prefix that consists of the terminal string “a” separated from the element by a colon “:”, that is, “a:e” or “a:(...)”. Thus the alphabet of L is $\{a, ':', e, '(', ')'\}$.

Here is a sample two-level list, with a main list of two elements and a sublist of one:

$$\begin{array}{c} \text{main list} \\ \boxed{a:(a:e) \ a:e} \\ \text{sublist} \end{array}$$

A *BNF* grammar G^s that generates the source language L is as follows (axiom S):

$$G^s \left\{ \begin{array}{l} S \rightarrow S P \\ S \rightarrow P \\ P \rightarrow a : E \\ E \rightarrow (S) \\ E \rightarrow e \end{array} \right.$$

Please answer the following questions and keep in mind that you *may not change* the source grammar G^s :

- (a) Write a *BNF* translation scheme (or grammar) G_1 that defines a translation function τ_1 that, for each element, turns the prefix into suffix. Example:

$$\tau_1 (a:(a:e) \ a:e) = (e:a):a \ e:a$$

Verify the correctness of scheme (or grammar) G_1 by drawing the translation tree of the example string.

- (b) Write a *BNF* translation scheme (or grammar) G_2 that defines a translation function τ_2 that, for each (sub)list, shifts the prefix of each (sub)list element and forms a chain of as many prefixes for the entire (sub)list. Example:

$$\tau_2 (a:e \ a:(a:e \ a:e) \ a:e) = a a a:e \ (a a:e e) \ e$$

Verify the correctness of scheme (or grammar) G_2 by drawing the translation tree of the example string.

- (c) (optional) Say if the schemes (or grammars) G_1 and G_2 found before are deterministic of type $LL(k)$ (or $ELL(k)$), for some $k \geq 1$, and explain why.

2. The grammar below (axiom S) generates (abstract syntax) trees with nodes of three types, that is, internal binary B , internal unary U and leaf f :

$$\left\{ \begin{array}{l} 1: S \rightarrow B \\ 2: B \rightarrow B B \\ 3: B \rightarrow U B \\ 4: B \rightarrow U U \end{array} \right. \quad \left\{ \begin{array}{l} 5: U \rightarrow U \\ 6: U \rightarrow B \\ 7: U \rightarrow f \end{array} \right.$$

A sample tree is shown on the next page.

For such trees we define the *depth* of a node of type B , U or f , to be the distance from the root node S minus one. For instance, in the sample tree the largest depth of the unary internal nodes (of type U) is 4.

We want to define an attribute grammar, based on the above syntax, to compute in the root node S the *number of internal unary nodes* that have *maximal depth*. This is the number of nodes of type U with a depth such that no other node of type U in the tree has a larger depth. In the sample tree, the number is 4 because there are four nodes of type U at depth 4, and no other node of type U at a larger depth.

Here are the attributes permitted to use (no other attributes are allowed):

<i>name</i>	<i>type</i>	<i>domain</i>	<i>symbol</i>	<i>meaning</i>
d	right	integer	B, U	depth ≥ 0 of a node
dp	left	integer	B, U	depth of the deepest node of type U that is a descendant of the current node; conventionally 0 if no such node exists
ndp	left	integer	S, B, U	number of nodes of type U , among the descendants of the current node, that have maximal depth; conventionally 0 if no such node exists

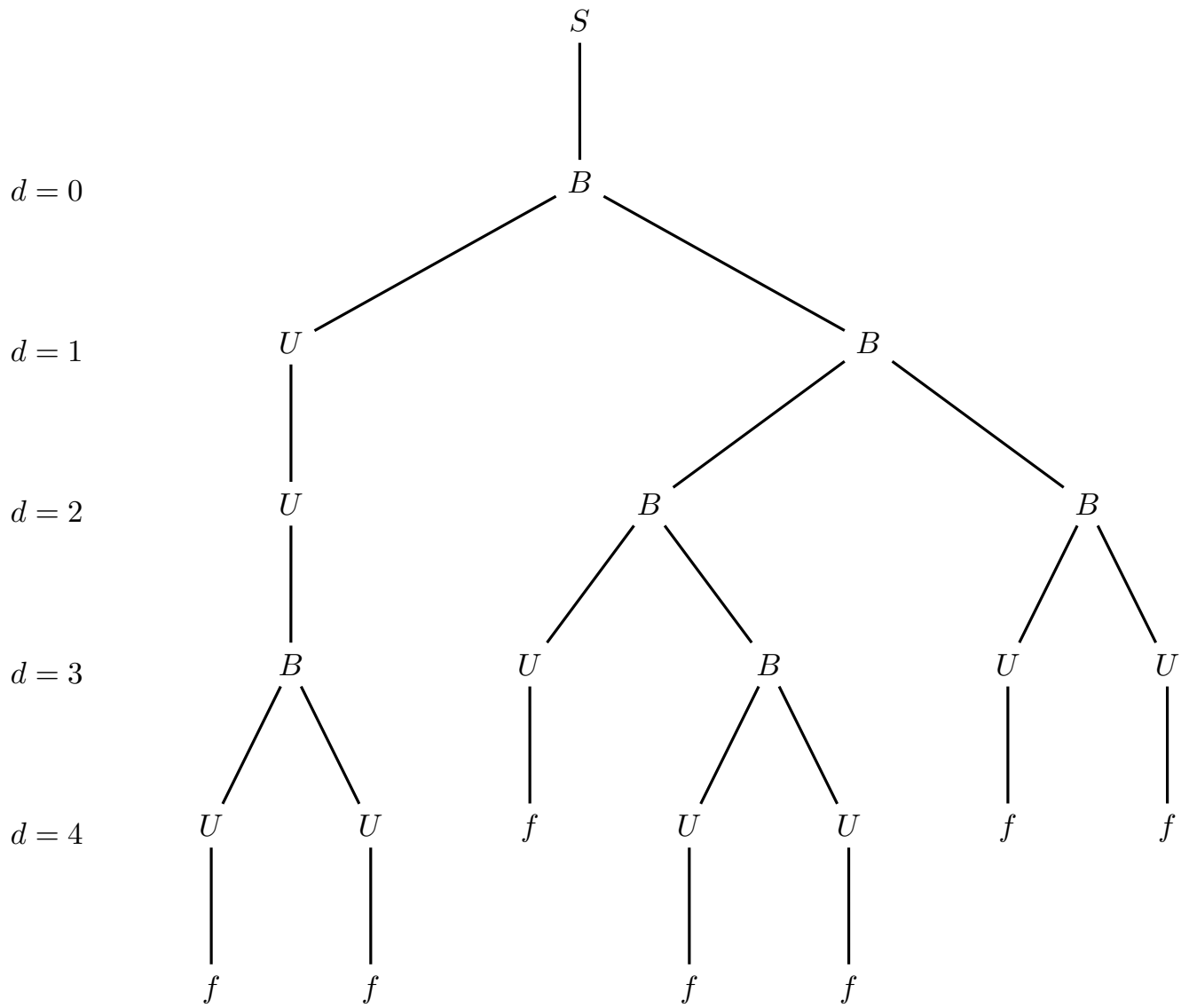
Answer the following questions (use the tables / trees / spaces on the next pages):

- Decorate the sample tree (next page) with the values of attributes dp and ndp .
- Write an attribute grammar, based on the syntax above and using only the permitted attributes (do not change the attribute types), that computes the required value of attribute ndp in the root node S of the tree. The attribute grammar must be of type one-sweep. Proceed stepwise and separately write:
 - the semantic rules for computing attribute d into table 1
 - those for attribute dp into table 2
 - and those for attribute ndp into table 3

Please, do not copy the rules already written in a table into the next one.

- (optional) Prove that your attribute grammar is of type one-sweep.

sample tree to decorate – question (a)



syntax

semantics – question (b) point (i) – TAB 1

1: $S_0 \rightarrow B_1$

2: $B_0 \rightarrow B_1 B_2$

3: $B_0 \rightarrow U_1 B_2$

4: $B_0 \rightarrow U_1 U_2$

5: $U_0 \rightarrow U_1$

6: $U_0 \rightarrow B_1$

7: $U_0 \rightarrow f$

syntax

semantics – question (b) point (ii) – TAB 2

1: $S_0 \rightarrow B_1$

2: $B_0 \rightarrow B_1 B_2$

3: $B_0 \rightarrow U_1 B_2$

4: $B_0 \rightarrow U_1 U_2$

5: $U_0 \rightarrow U_1$

6: $U_0 \rightarrow B_1$

7: $U_0 \rightarrow f$

syntax

semantics – question (b) point (iii) – TAB 3

1: $S_0 \rightarrow B_1$

2: $B_0 \rightarrow B_1 B_2$

3: $B_0 \rightarrow U_1 B_2$

4: $B_0 \rightarrow U_1 U_2$

5: $U_0 \rightarrow U_1$

6: $U_0 \rightarrow B_1$

7: $U_0 \rightarrow f$

please here answer question (c)

free space left for the continuation of any question of any exercise